Semaphores
Part II

Sistemas de Computadores
Departamento de Engenharia Informática
Instituto Superior de Engenharia do Porto

Luís Nogueira (lmn@isep.ipp.pt)

Recall from last class

- **`sem_open()`**
  - Create/open a POSIX named semaphore
- **`sem_post()`**
  - Increment (up) the semaphore's value
- **`sem_wait()`, `sem_trywait()`, `sem_timedwait()`**
  - Try to decrement (down) the semaphore's value
  - `sem_wait()` blocks; `sem_trywait()` does not block; `sem_timedwait()` blocks with timeout
- **`sem_getvalue()`**
  - Get the current semaphore's value

Recall from last class

- **`sem_close()`**
  - Releases the process's reference to the semaphore

- **`sem_unlink()`**
  - Removes the semaphore name from the system
  - The semaphore is destroyed only after it has been unlinked and all processes have closed it

Beyond simple mutual exclusion

- **In the last class, we used semaphores to solve one specific problem:**
  - Protecting a critical section, ensuring mutual exclusion to prevent race conditions
- **Mutual exclusion solves:**
  - Who can enter the critical section?
  - How do we prevent concurrent writes?
- **But semaphores are more than "mutexes with a different name"**

Beyond simple mutual exclusion

- **Mutual exclusion does not solve:**
  - How do we enforce execution order?
  - How do we wait for an event?
  - How do we coordinate producers and consumers?
  - How do we synchronize multiple stages of a computation?

- **Understanding this distinction is essential for designing correct concurrent programs**

Beyond simple mutual exclusion

- **In this class, we will explore how semaphores can be used to solve a set of new problems**

- **Event signaling**
  - Using semaphores to signal the occurrence of events

- **Check-Then-Act problem**
  - Synchronizing actions based on conditions

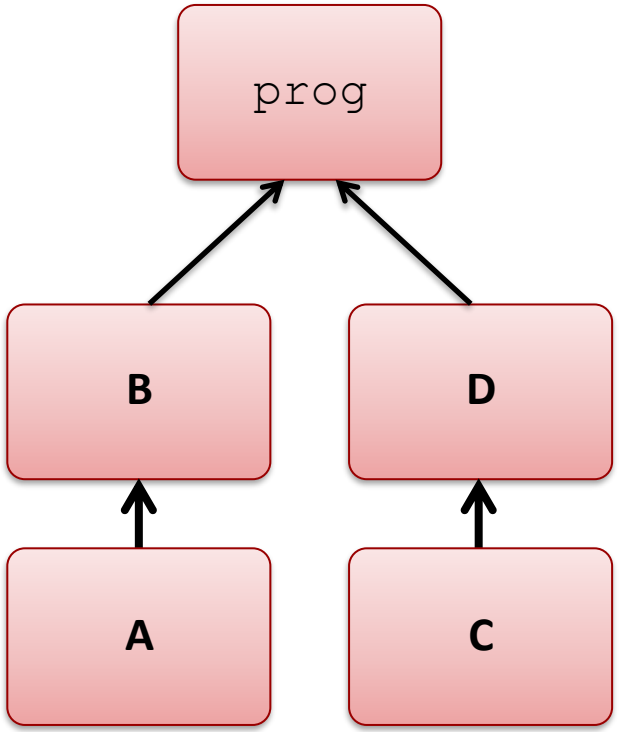
- **Synchronization barrier**
  - Coordinating multiple processes to reach a specific point before continuing

Exercise 1 – Event signaling

- **This exercise focuses on signaling the occurrence of events, not on protecting shared data**
  - There is no shared critical section to protect
- **Instead, we must enforce execution order constraints**
- **How must the semaphores be initialized to enforce the required ordering?**
- **At what exact points in the execution should `sem_wait()` and `sem_post()` be invoked?**

Exercise 1 – Event signaling

- **Implement a program that creates 4 child processes to compile 4 objects (A,B,C,D) to generate an executable file `prog`**
  - Objects are compiled with `execlp()` with command `make [A, B, C, D]`
  - Object B depends on A and object D depends on C
  - All 4 children are created immediately
  - Parent waits for children A and C to terminate correctly and signals to B and D **using semaphores**
  - Parent creates the final executable with `make prog`



Exercise 2 – Check-Then-Act

- **This exercise focuses on a classic concurrency error**
  - Checking a condition and acting on it **must be atomic**

```
/* the availability check is performed outside the critical section */

if(shm->tickets > 0){
    sem_wait(sem_excl);
    shm->tickets--;
    sem_post(sem_excl);
}
```

- **We are not just protecting data**
  - Between the check and the decrement, another process may modify tickets, and the condition may no longer hold
  - We must ensure that a decision based on shared state remains valid when we act on it

Exercise 2 – Check-Then-Act

- **The parent must:**
  - Create a shared memory region containing:
    - An integer tickets, initialized to 20,000
    - An array of 5 counters (one per child), each initialized to 0
  - Create a semaphore to protect access to the shared memory
  - Fork 5 child processes
  - Wait for all children to terminate
  - After all children finish, print the number of tickets bought by each child

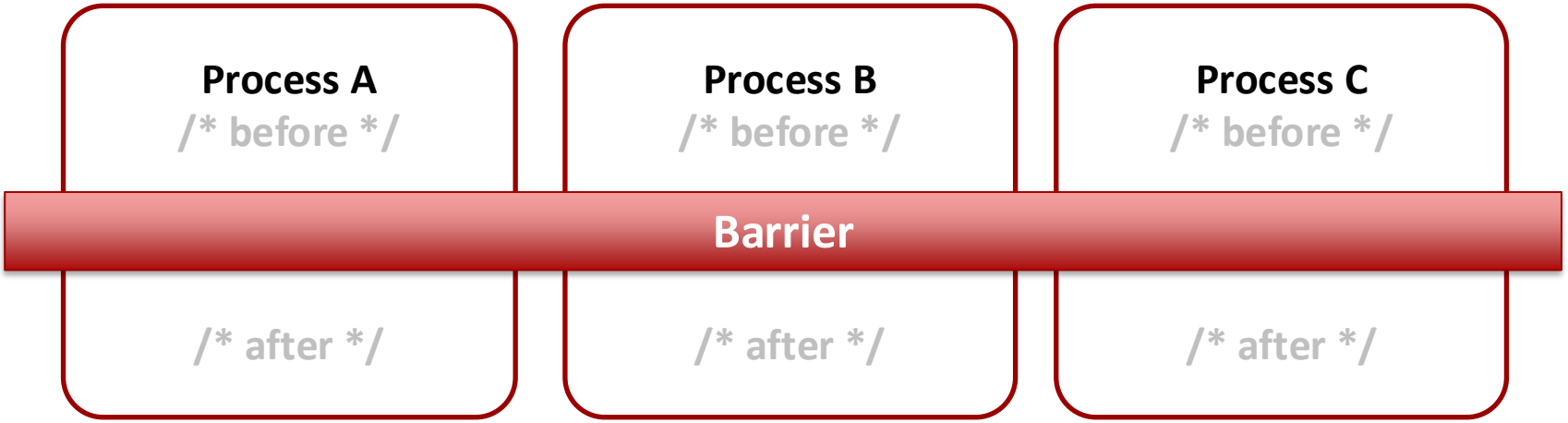
Exercise 2 – Check-Then-Act

- **Each child must:**
  - Repeatedly attempt to buy tickets while tickets remain available
  - Use the semaphore to ensure mutual exclusion when accessing shared memory
  - Check whether tickets are available
  - Decrement the number of available tickets
  - Increment its own counter in shared memory
  - Terminate when no tickets remain

- **The check and the action must be executed atomically to avoid a check–then–act race condition**

Exercise 3 — Barrier synchronization

- **This exercise focuses on coordinating multiple processes that must progress in phases**
  - The goal is not only mutual exclusion — it is **ensuring that all processes reach a certain point before any can continue**



Example – barrier

- **Use multiple child processes to calculate partial sums of random numbers stored in separate arrays and update a shared total sum**

- **Each child:**
  - Fills one unique array with 1000 random numbers
  - Child 1 fills `v1[]`; Child 2 fills `v2[]`; Child 3 fills `v3[]`
  - Calculates the partial sum ($\sum$ `v1[j]+v2[j]+v3[j]`) of 1/3 of the arrays
  - Sums the computed partial to a total sum in shared memory

- **Note that all children need to fill in the respective array before computing their partial sum**

Implementing a barrier

- **This implementation works for a single barrier point**
  - Not reusable safely for multiple barrier cycles
  - Classic reusable barrier requires two-phase turnstile

```
/* semaphore to guarantee mutual exclusion to a counter in shared memory of processes waiting at the barrier */
sem_nproc = sem_open(”/nproc", O_CREAT|O_EXCL, 0644, 1);

/* semaphore to have processes blocked at the barrier */
sem_barrier = sem_open(”/barrier", O_CREAT|O_EXCL, 0644, 0);

/* shared data */
typedef struct{
   int v1[N], v2[N], v3[N];
   int total_sum;
   int nproc_at_barrier;
}shared_data_t;
```

Implementing a barrier

```
/* increases counter of processes waiting at the barrier */
sem_wait(sem_nproc);
shm->nproc_at_barrier++;

/* checks if all processes have reached the barrier */
/* the last arriving process releases the barrier once */
if(shm->nproc_at_barrier == NPROC)
   sem_post(sem_barrier);

sem_post(sem_nproc);

/* waits in barrier */
sem_wait(sem_barrier);

/* each passing process re-posts it so that the next process may continue */
sem_post(sem_barrier);

/* calculate partial sum */
…
```

Exercise 3 — Barrier synchronization

- **Implement a program that counts the number of occurrences of the maximum value in an array filled with random values**

- **Create a shared memory area**
  - With two integers: `gmax` (global maximum value) and `max_count` (occurrences of `gmax`)
  - Initialize an array `v[1000]` with random numbers

- **Create 10 children that execute concurrently**
  - Each processes 1/10 of `v[]` and determines the maximum value in `v[]` and updates `gmax`
  - When `gmax` is known, each child should update `max_count` with the number of occurrences of `gmax` in its 1/10 of `v[]`